In these slides

Boundary value ODEs

In Computational Physics 1 we looked at how to solve boundary value problems and eigenvalue problems in ODEs. There are two main methods:

Shooting method

1. Guess unknown initial values v_i
2. Solve ODE with these values: $f(x|v_i)$
3. Find solution at final point x_f
4. Solve $f(x_f|v_i) - v_f = 0$ using root finding methods.

Relaxation method

1. Guess entire solution satisfying boundary conditions
2. 'Relax' trial solution to actual solution

Eigenvalue problems may be made into boundary value problem by treating the eigenvalue as an additional variable.

Matrix method

For some ODEs, there is another technique:

Transform equation to matrix equation

Example

Consider the equation

$$y''(x) = f(x)\,, \qquad y(a) = Y_a, \quad y(b) = Y_b\,.$$

Divide $[a, b]$ into N sub-intervals, with $N + 1$ equally spaced points.

$$x_i = a + i\delta\,, \quad y_i = y(x_i) \quad \delta = \frac{b-a}{N}\,, \quad i = 0, \ldots, N\,.$$

Replacing $y''(x)$ with discrete derivative, we get

$$\frac{y_{i+1} - 2y_i + y_{i-1}}{\delta^2} = f(x_i)\,, \quad y_0 = Y_a, \;\; y_N = Y_b\,.$$

A system of linear equations, i.e., a matrix equation.

Matrix equation

Obtained system of linear equations:

$$y_{i-1} - 2y_i + y_{i+1} = \delta^2 f(x_i) \equiv \hat{f}_i\,, \quad i = 1, \ldots, N-1\,. \qquad (*)$$

We can write this in matrix form

$$\begin{pmatrix} -2 & 1 & 0 & \ldots\ldots & 0 \\ 1 & -2 & 1 & 0 \cdots & 0 \\ 0 & 1 & -2 & 1 \cdots & 0 \\ \ldots\ldots & \ldots\ldots & \ldots\ldots & \ldots\ldots & \ldots\ldots \\ 0 & \ldots\ldots & \ldots\ldots & \ldots\ldots & -2 \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_{N-1} \end{pmatrix} = \begin{pmatrix} \hat{f}_1 - Y_a \\ \hat{f}_2 \\ \hat{f}_3 \\ \vdots \\ \hat{f}_{N-1} - Y_b \end{pmatrix}$$

Boundary conditions enter first and last equations:

$$y_0 - 2y_1 + y_2 = \hat{f}_i \qquad \Longrightarrow \qquad -2y_1 + y_2 = \hat{f}_1 - y_0 = \hat{f}_1 - Y_a$$

Matrix equation

Discretized to $N+1$ points, with $N-1$ interior points.
The boundary values of $y(x)$ are known (Dirichlet boundary conditions).

$\Longrightarrow$ $(N+1) \times (N+1)$ matrix.

Could solve by inverting matrix:

$$\begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_{N-1} \end{pmatrix} = \begin{pmatrix} -2 & 1 & 0 & \cdots\cdots & 0 \\ 1 & -2 & 1 & 0 \cdots & 0 \\ 0 & 1 & -2 & 1 \cdots & 0 \\ \cdots\cdots & \cdots\cdots & \cdots\cdots & \cdots\cdots & \cdots\cdots \\ 0 & \cdots\cdots & \cdots\cdots & \cdots\cdots & -2 \end{pmatrix}^{-1} \begin{pmatrix} \hat{f}_1 - Y_a \\ \hat{f}_2 \\ \hat{f}_3 \\ \vdots \\ \hat{f}_{N-1} - Y_b \end{pmatrix}$$

Calculating inverse of matrix explicitly $\longrightarrow$

- inefficient
- usually unnecessary
- but works for moderate N.

Neumann boundary conditions

The boundary conditions were: $y(a) = Y_a$, $y(b) = Y_b$
Values of function given at boundary $\longrightarrow$ Dirichlet boundary conditions

If Derivatives given at boundary? $\longrightarrow$ Neumann boundary conditions
E.g., $y'(a) = \xi_a$ given instead of $y(a)$.

We used for first equation:

$$y_0 - 2y_1 + y_2 = \hat{f}_1 \qquad \Longrightarrow \qquad -2y_1 + y_2 = \hat{f}_1 - y_0 = \hat{f}_1 - Y_a$$

No longer works, y_0 not known. Need additional equation for y_0.

Use a finite difference formula for $y'(x)$:

Option 1: forward difference:

$$\xi_a = (y_1 - y_0)/\delta$$
$$\Longrightarrow \qquad -y_1 + y_2 = \hat{f}_1 + \delta\xi_a$$

Problem: an $O(\delta)$ approximation.

Destroys $O(\delta^2)$ accuracy of complete procedure

Neumann boundary conditions

$y'(a) = \xi_a$ given. For the first equation, $y_0 - 2y_1 + y_2 = \hat{f}_1$, we need additional equation for y_0.

Use a finite difference formula for $y'(x)$:

Option 2: Use a second-order forward difference: $$\xi_a = \frac{4y_1 - y_2 - 3y_0}{2\delta}$$

Option 3: Use a second-order centred difference: $$\xi_a = \frac{y_1 - y_{-1}}{2\delta}$$

Problem: a fictional external point ($x_{-1} = a - i\delta$) is introduced. Need equation for y_{-1} as well. Can use

$$y_{-1} - 2y_0 + y_1 = \hat{f}_0$$

Linear algebra

Starting from a boundary value problem we ended up with a linear algebra problem!

$$Ay = b \quad \overset{\text{formally}}{\Longleftrightarrow} \quad y = A^{-1}b$$

The problem is 'equivalent' to inverting the matrix A

Matrix problems appear in

- solving sets of linear equations
- static solutions of pdes
- quantum mechanics: single-particle, many-particle, many-spin,...
- nonlinear or correlated curve fitting
- .....

Related problems: calculating eigenvalues and eigenvectors, eg $H\Psi = E\Psi$

Methods

- Direct elimination methods
  - Gauss–Jordan, LU decomposition, QR, Cholesky
  - Works with all kinds of matrices but best for small
- Iteration
  - Jacobi, Gauss–Seidel, overrelaxed Gauss–Seidel
  - Write $Ax = (E - F)x = b$ where E is easily invertible
  - Iterate $x^{(n+1)} = E^{-1}(Fx^{(n)} + b)$
  - Requires diagonally dominant matrices, can be arbitrarily large
  - Later!
- Conjugate gradient or similar
  - Very useful for sparse matrices
  - Does not require A to be known explicitly, only the vector multiplication $y = Ax$
  - Can be used with any sparse matrix of arbitrary size

Numerical issues

- Storage
  - matrices from discretising physical systems can be huge
- Speed
  - Direct elimination scales like N^3
- Singularity
  - Direct elimination methods become unstable
  - Iterative methods can slow down indefinitely
  - Singular value decomposition works if matrix not too large
  - Governed by condition number $\lambda_{\text{max}}/\lambda_{\text{min}}$
  - Preconditioning!

Gaussian elimination

Probably the one method you already know!

We want to find the x_i in the equation(s)

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \end{bmatrix}$$

We can

- interchange any two rows of A, b
- replace any row by a linear combination of itself and another
- interchange columns of A and the corresponding rows of x

The most naïve method uses just the second operation!
First and third: pivoting

Without pivoting

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \end{bmatrix}$$

$$\begin{bmatrix} 1 & a_{12}/a_{11} & a_{13}/a_{11} & a_{14}/a_{11} \\ 0 & 1 & \cdot & \cdot \\ 0 & 0 & 1 & \cdot \\ 0 & 0 & 0 & \tilde{a}_{44} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} b_1/a_{11} \\ \tilde{b}_2 \\ \tilde{b}_3 \\ \tilde{b}_4 \end{bmatrix}$$

Now we have the matrix in upper triangular form and can obtain all the x_i by backsubstitution:

$$x_4 = \tilde{b}_4/\tilde{a}_{44}, \qquad x_3 = \tilde{b}_3 - \tilde{a}_{34}x_4$$

etc

This goes horribly wrong if we get a zero on the diagonal!
It is also unstable with any rounding error.
Pivoting: swap rows so largest available element appears on diagonal

Sparse matrices

We had the equation

$$y_{i+1} - 2y_i + y_{i-1} = f_i \Longrightarrow \mathbf{Ay} = \mathbf{f} \quad \text{with} \quad A_{ij} = \delta_{i,j-1} - 2\delta_{ij} + \delta_{i,j+1}$$

This is a tridiagonal matrix
— a very common type in physical and mathematical problems
Other common types of matrices:

- band diagonal (with bandwidth M)
- tridiagonal with fringes (eg the two-dim Laplace operator)
- cyclic tridiagonal or banded (with fringes)
- band triangular
- block diagonal
- block tridiagonal
- block triangular
- singly/doubly bordered block diagonal
- other

Sparse matrices

Savings possible when matrix is sparse

- massive saving in storage
  - only store nonzero elements and some bookkeeping
  - Various formats are in use, c.f. NR 2.7
- massive saving in number of compute operations
  - reduced from N^3 to N^2
- often you need not store the matrix at all
  - supply function $y(x) = Ax$
  - same function for different matrix sizes!

Sparse matrices in Python

Scipy provides data structures and routines for dealing with sparse matrices in the scipy.sparse package. These can imported with:

```
import scipy.sparse as sparse
```

There are various different types for sparse matrix available.

- bsr matrix: Block Sparse Row matrix
- coo matrix: A sparse matrix in COOrdinate format.
- csc matrix: Compressed Sparse Column matrix
- csr matrix: Compressed Sparse Row matrix
- dia matrix: Sparse matrix with DIAgonal storage
- dok matrix: Dictionary Of Keys based sparse matrix.
- lil matrix: Row-based linked list sparse matrix
- spmatrix: This class provides a base class for all sparse matrices.

Sparse matrices in Python

Some matrices are better for constructing and others better for computation.

- Construction: coo matrix, dok matrix, lil matrix
- Computation: bsr matrix, csc matrix, csr matrix

Example of construction with coo matrix

```
row = np.array([0, 3, 1, 0])
col = np.array([0, 3, 1, 2])
data = np.array([4.0, 5.0, 7.0, 9.0])
A = sparse.coo matrix((data, (row, col)), shape=(4, 4)))
print(A)
```

gives:
(0, 0) 4
(3, 3) 5
(1, 1) 7
(0, 2) 9

Sparse matrices in Python

Other useful operations:

- Convert to dense matrix using todense.
- Convert to csr or csc using tocsr and tocsc for fast arithmetic.
- Look at non zeros using plt.spy from matplotlib.

Given a matrix A and a vector b, we wish to find an x that solves:

$$Ax = b$$

When A and b are dense we use solve from numpy:

```
import numpy as np
x = np.linalg.solve(A,b)
```

When A and b are sparse we use solve from scipy:

```
import scipy.sparse as sparse
x = sparse.linalg.spsolve(A,b)
```

Sparse matrices in Python

For constructing banded matrices spdiags is very useful:

Example

```
import scipy.sparse as sparse
import numpy as np
data = -np.ones((3,4))
data[1,:] *= -2
A = sparse.spdiags(data, [-1,0,1], 4, 4)
print(A.todense())
```

Summary

- Many boundary-value problems can be discretised, made into matrix equations
- Three general methods for solving matrix equation $Ax = b$:
  - Direct elimination (Gauss–Jordan etc)
  - Iteration (Gauss–Seidel etc) for diagonally dominant matrices
- Sparse matrices appear in many physical problems
  - Huge savings in storage and computation
  - Many numerical methods are tailor-made for sparse matrices